



Python Programming Fundamentals

JSON Persistence

Programming Fundamentals Team

SETU

2026-06-12



Outline

1. What is JSON?
2. `json.dumps()` and `json.loads()`
3. `json.dump()` and `json.load()`
4. Saving Product Objects
5. Loading Product Objects Back
6. Full Shop Save and Load
7. Integrating with the Menu



Outline

1. What is JSON?
2. `json.dumps()` and `json.loads()`
3. `json.dump()` and `json.load()`
4. Saving Product Objects
5. Loading Product Objects Back
6. Full Shop Save and Load
7. Integrating with the Menu



1.1 JavaScript Object Notation

JSON is a **text format** for storing and exchanging structured data. It is:

- Human-readable
- Language-independent
- The most common data format for web APIs and config files

```
{  
  "name": "Apple",  
  "price": 1.99,  
  "product_id": 101  
}
```

Python maps: dict ↔ JSON object, list ↔ JSON array, str/int/float/bool/None ↔ their JSON equivalents.



Outline

1. What is JSON?
2. `json.dumps()` and `json.loads()`
3. `json.dump()` and `json.load()`
4. Saving Product Objects
5. Loading Product Objects Back
6. Full Shop Save and Load
7. Integrating with the Menu



2.1 Strings ↔ Python objects

```
import json

# Python dict → JSON string
data = {"name": "Apple", "price": 1.99, "id": 101}
json_str = json.dumps(data, indent=2)
print(json_str)
# {
#   "name": "Apple",
#   "price": 1.99,
#   "id": 101
# }

# JSON string → Python dict
loaded = json.loads(json_str)
print(loaded["name"])    # Apple
print(type(loaded))      # <class 'dict'>
```



Outline

1. What is JSON?
2. `json.dumps()` and `json.loads()`
3. `json.dump()` and `json.load()`
4. Saving Product Objects
5. Loading Product Objects Back
6. Full Shop Save and Load
7. Integrating with the Menu



3.1 Files ↔ Python objects

```
import json

data = [{"name": "Apple", "price": 1.99, "id": 101},
        {"name": "Bread", "price": 2.50, "id": 102}]

# Write to file
with open("products.json", "w") as f:
    json.dump(data, f, indent=2)

# Read from file
with open("products.json", "r") as f:
    loaded = json.load(f)

print(loaded[0]["name"])    # Apple
```



Outline

1. What is JSON?
2. `json.dumps()` and `json.loads()`
3. `json.dump()` and `json.load()`
- 4. Saving Product Objects**
5. Loading Product Objects Back
6. Full Shop Save and Load
7. Integrating with the Menu



4.1 Using __dict__

```
# Each Product object has a __dict__ – but with mangled names
# Better: add a to_dict() method to Product
```

```
class Product:
    def to_dict(self) -> dict:
        return {
            "name": self.__name,
            "price": self.__price,
            "product_id": self.__product_id,
        }
```

```
# Saving
products = shop.get_all_products()
data = [p.to_dict() for p in products]
with open("shop.json", "w") as f:
```



4.1 Using `__dict__`

```
json.dump(data, f, indent=2)
print(f"Saved {len(data)} products.")
```



Outline

1. What is JSON?
2. `json.dumps()` and `json.loads()`
3. `json.dump()` and `json.load()`
4. Saving Product Objects
- 5. Loading Product Objects Back**
6. Full Shop Save and Load
7. Integrating with the Menu



5.1 Using a class method

```
class Product:
    @classmethod
    def from_dict(cls, d: dict) -> "Product":
        return cls(d["name"], d["price"], d["product_id"])

# Loading
def load_shop(shop: Shop, filename: str):
    try:
        with open(filename, "r") as f:
            data = json.load(f)
        for item in data:
            shop.add_product(Product.from_dict(item))
        print(f"Loaded {len(data)} products.")
    except FileNotFoundError:
        print("No saved data found – starting fresh.")
```



5.1 Using a class method

```
except (json.JSONDecodeError, KeyError) as e:  
    print(f"Error loading data: {e}")
```



Outline

1. What is JSON?
2. `json.dumps()` and `json.loads()`
3. `json.dump()` and `json.load()`
4. Saving Product Objects
5. Loading Product Objects Back
- 6. Full Shop Save and Load**
7. Integrating with the Menu



6. Full Shop Save and Load

```
# In shop.py
import json
from product import Product

class Shop:
    def save_to_json(self, filename: str = "shop.json"):
        data = {
            "name": self.__name,
            "products": [p.to_dict() for p in self.__products]
        }
        with open(filename, "w") as f:
            json.dump(data, f, indent=2)

    def load_from_json(self, filename: str = "shop.json"):
        with open(filename, "r") as f:
            data = json.load(f)
```



6. Full Shop Save and Load

```
self.__name = data["name"]
self.__products = [
    Product.from_dict(p) for p in data["products"]
]
```



Outline

1. What is JSON?
2. `json.dumps()` and `json.loads()`
3. `json.dump()` and `json.load()`
4. Saving Product Objects
5. Loading Product Objects Back
6. Full Shop Save and Load
- 7. Integrating with the Menu**



7. Integrating with the Menu

```
def main():
    shop = Shop("My Shop")
    # Load on startup
    try:
        shop.load_from_json()
        print("Previous data loaded.")
    except FileNotFoundError:
        print("Starting with empty shop.")

    while True:
        show_menu()
        match input("Choice: ").strip():
            # ... CRUD cases ...
            case "s":
                shop.save_to_json()
                print("Saved.")
```



7. Integrating with the Menu

```
case "0":  
    shop.save_to_json()    # auto-save on exit  
    print("Data saved. Goodbye!")  
    break
```

Thanks for Watching - Any questions?

